Понятия ООП

Наследование, инкапсуляция, полиморфизм

Корнеева Елена
https://t.me/Allyonzy

2025

План занятия (лекция + семинар)

Принципы ООП в Python

1. Классы в Python. Поля, методы, уровни доступа
2. Принципы ООП: инкапсуляция, наследование, полиморфизм



Классы в Python

- Классы
- Поля
- Методы
- Уровни доступа



Виды программирования

Программирование можно классифицировать по различным парадигмам, каждая из которых имеет свои особенности и применения. Вот ***основные виды*** программирования.

Структурное программирование

- Парадигма, которая организует код в виде иерархической структуры блоков, используя базовые управляющие конструкции: последовательность, ветвление и цикл.
- Добавление коду структуры

Процедурное программирование

- Основой программы является функция. Функции вызывают друг друга и при необходимости передают данные. В программе функции живут отдельно, данные — отдельно.
- Основной недостаток - сложность создания и поддержки больших программ. Наличие сотен функций в таких проектах очень часто приводит к ошибкам и спагетти-коду.

Функциональное программирование

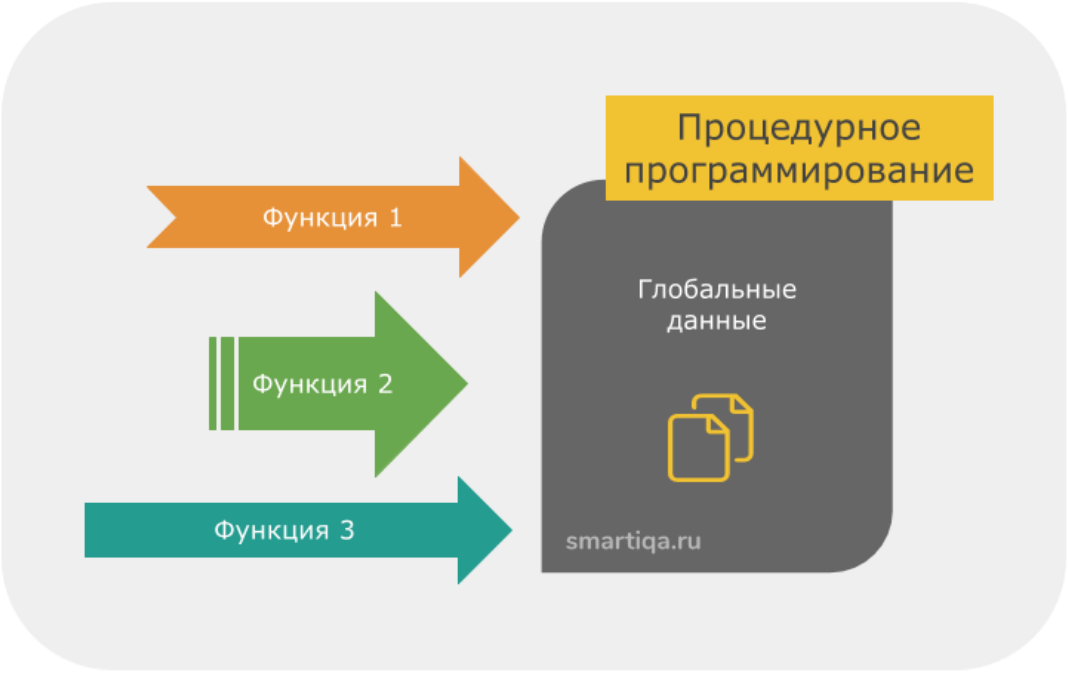
- Парадигма, основанная на использовании функций как основных строительных блоков программы.
- Избежать изменения состояния и побочных эффектов, что делает код более предсказуемым и тестируемым.

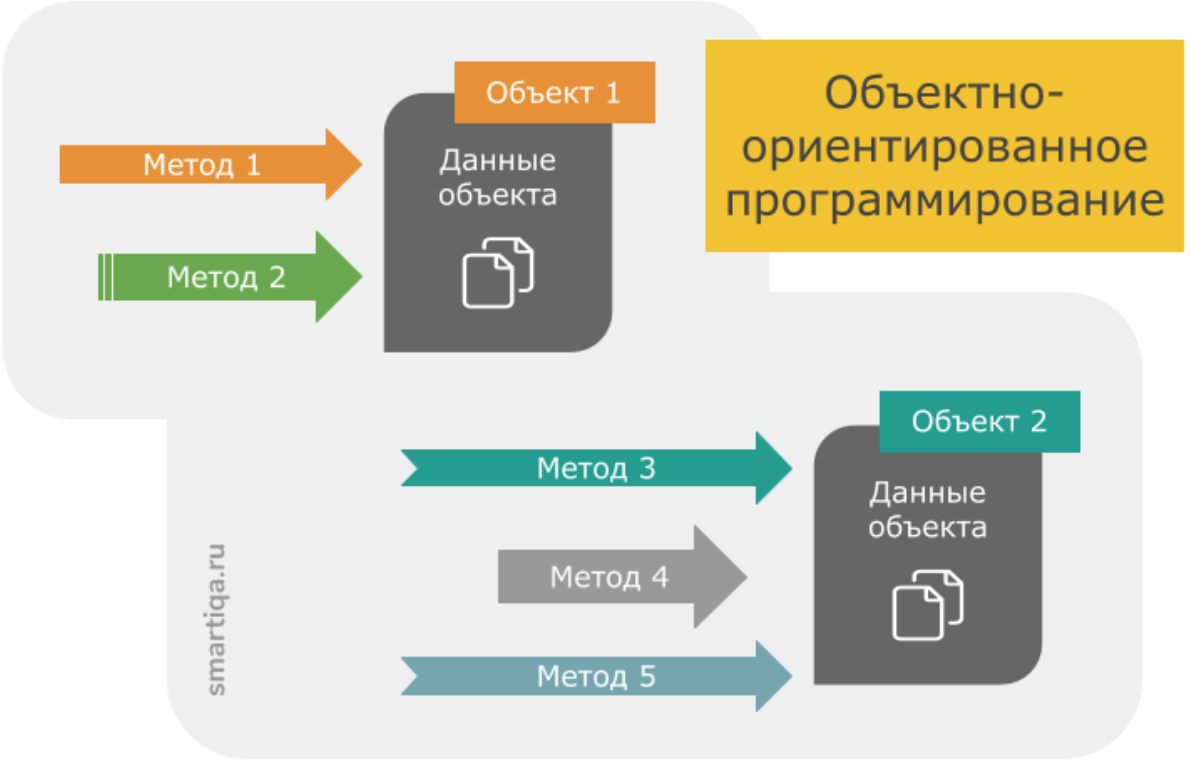
Объектно-ориентированное программирование (ООП)

- В основе лежит понятие объекта. Объект совмещает в себе и функции и данные.
- Основное преимущество - изоляция кода на уровне классов, что позволяет писать более простой и лаконичный код.

Что такое ООП?

Основные преимущества





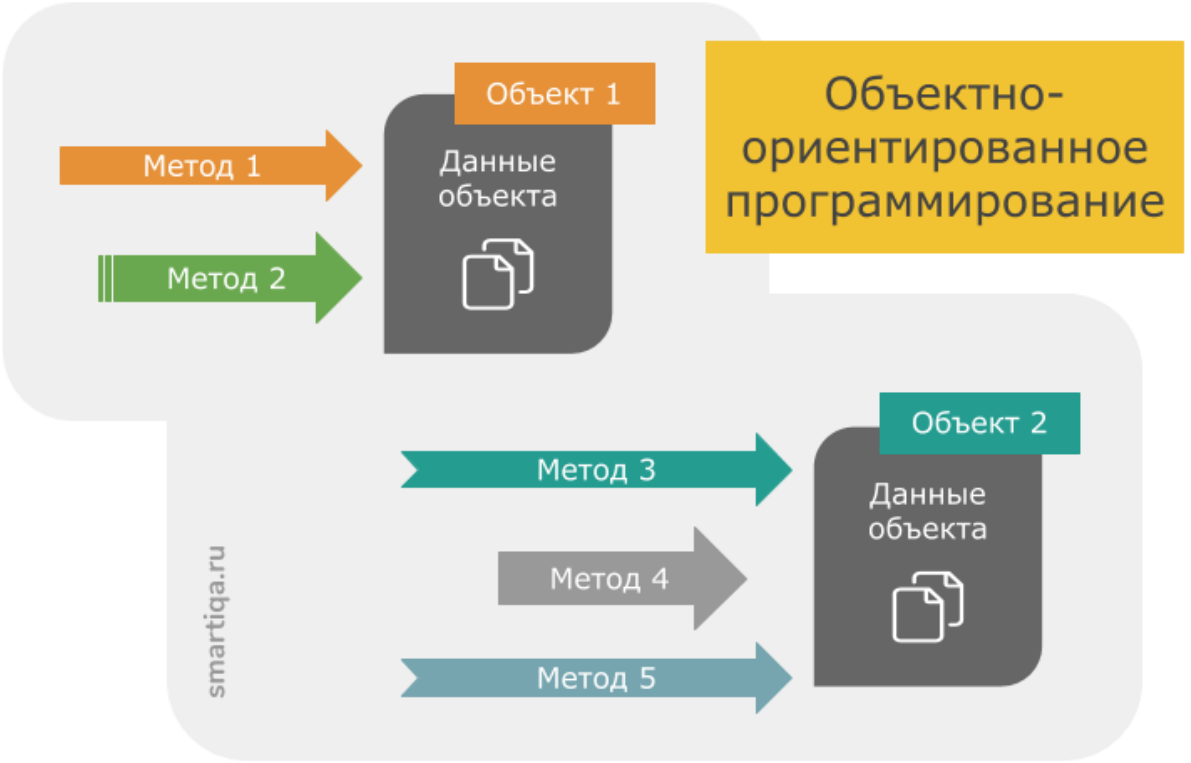
Процедурное программирование – это написание функций и их последовательный вызов в некоторой главной(main) функции.

Объектно-ориентированное программирование (ООП) — методология программирования, основанная на представлении программы в виде совокупности объектов, каждый из которых является экземпляром определённого класса, а классы образуют иерархию наследования.

Объектно-ориентированное программирование (ООП)

Главное в программе - это **данные**. Данные определяют, какие методы будут использоваться для их обработки. Т. е. они первичны, код для обработки этих данных - вторичен.

1. ***Программа разбивается на объекты.*** Каждый объект отвечает за собственные данные и их обработку. Как результат - код становится проще и читабельней.
2. ***Уменьшается дупликация кода***. Нужен новый объект, содержимое которого на 90% повторяет уже существующий? Давайте создадим новый класс и унаследуем эти 90% функционала от родительского класса!
3. ***Упрощается и ускоряется процесс написания программ.*** Можно сначала создать высокоуровневую структуру классов и базовый функционал, а уже потом перейти к их подробной реализации.



Объектно-ориентированное программирование (ООП)

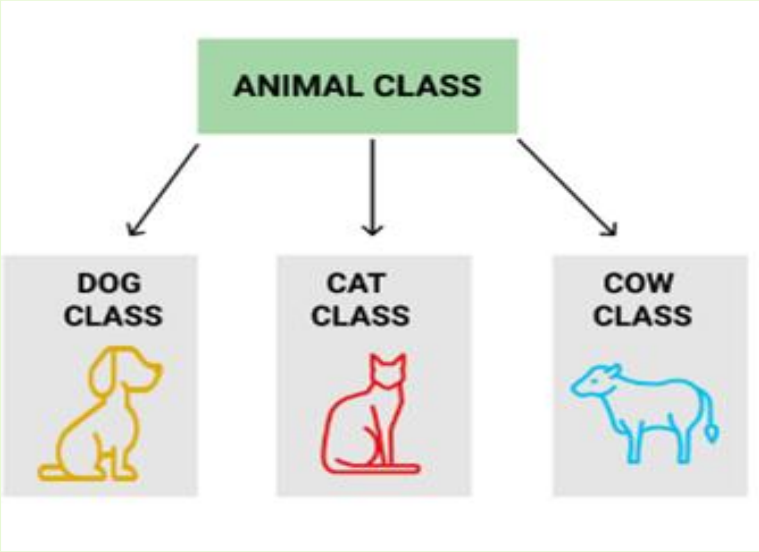
ООП – это метод разработки больших программ!



- основная программа – простая и понятная
- классы могут разрабатывать разные программисты независимо друг от друга (+интерфейс!)
- повторное использование классов



- неэффективно для небольших задач



Краткий словарь программиста: часть 1

Спагетти-код — это плохо спроектированная, слабо структурированная, запутанная и трудная для понимания программа. Содержит много операторов GOTO (как правило, переходов назад), исключений и других конструкций, ухудшающих структурированность.

Процедурное программирование – это написание функций и их последовательный вызов в некоторой главной(main) функции.

Объектно-ориентированное программирование (ООП) — методология программирования, основанная на представлении программы в виде совокупности объектов, каждый из которых является экземпляром определённого класса, а классы образуют иерархию наследования.

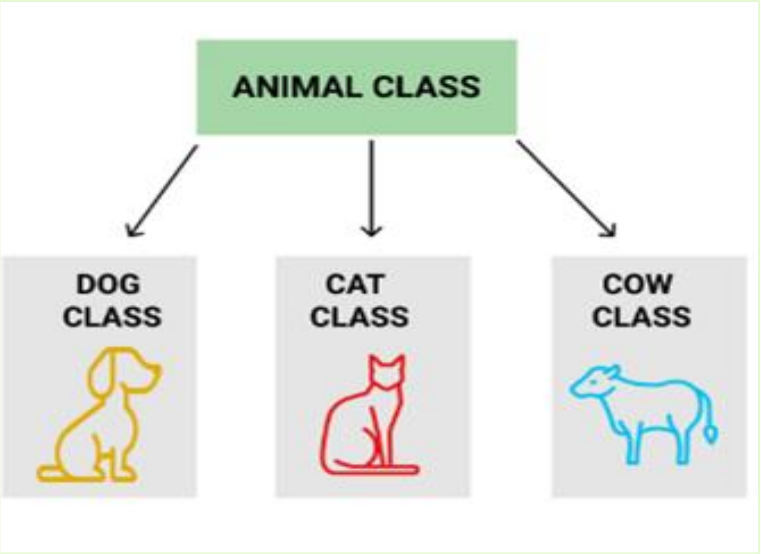
Класс — в объектно-ориентированном программировании, представляет собой шаблон для создания объектов, обеспечивающий начальные значения состояний: инициализация полей-переменных и реализация поведения функций или методов.

Объект — некоторая сущность в цифровом пространстве, обладающая определённым состоянием и поведением, имеющая определенные свойства (поля) и операции над ними (методы). Как правило, при рассмотрении объектов выделяется то, что объекты принадлежат одному или нескольким классам, которые определяют поведение (являются моделью) объекта. Термины «экземпляр класса» и «объект» взаимозаменяемы.



Объекты и классы в ООП

ООП **–** это метод разработки больших программ!



- Классы в Python представляют **шаблоны** или **определения** для **создания объектов определенного типа**.
- Класс определяет **состояние** и **поведение** объектов, а объект является экземпляром класса.
- Для определения класса используется **ключевое слово class**, за которым следует имя класса (обычно с заглавной буквы).
- Создание объекта класса осуществляется путем вызова **конструктора** класса с использованием имени класса, за которым следуют круглые скобки.

```
class = Team { city: varchar,
               name: varchar,
               coach: varchar,
               arena: varchar}
```

Экземпляр класса →

city	name	coach	arena
Казань	БК УНИКС	Иванов	БСК УНИКС

Объекты и классы в ООП

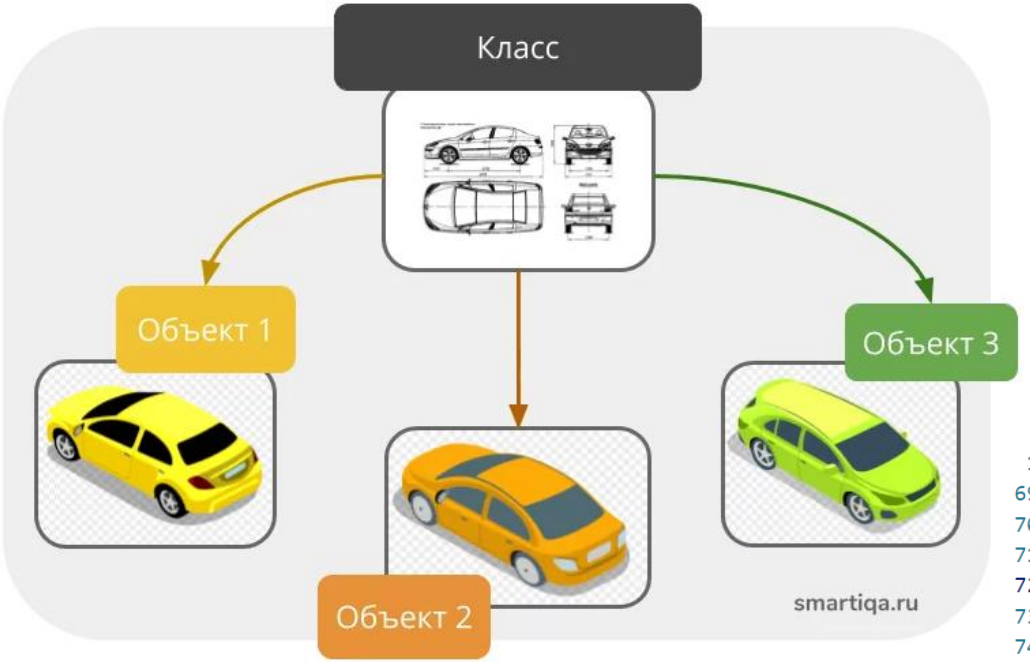
Класс «Автомобиль»

Свойства класса:

- Цвет
- Объем двигателя
- Мощность
- Тип коробки передач

Методы класса:

- Ехать
- Остановиться
- Заправиться
- Поставить на сигнализацию
- Включить дворники



Что общего будет в объектах?

- по одному шаблону
- будут обладать одинаковыми

В чем разница?

- значения свойств

```python
class Automobile: ...

def main():
    # позволяет разделить код, который выполняется
    # и код, который импортируется как модуль
    # пример использования
    auto = Automobile("Red", 2.0, 150, "Automatic")
    print(auto)
    auto.drive()
    auto.stop()
    auto.refuel()
    auto.activate_alarm()
    auto.turn_on_wipers()

    auto.power = 160
    print(auto)

if __name__ == "__main__":
    main()
```

Классы и объекты в Python

Синтаксис: создание класса

```
class <название_класса>:
    <тело_класса>
```

```
class Car:
    pass
```

Синтаксис: создание экземпляра

```
<имя_объекта> = <имя_класса>()
```

```
car_object = Car()
```

Краткий словарь программиста: часть 2

Класс описывает множество объектов, имеющих общую структуру и обладающих одинаковым поведением. ***Класс*** - это шаблон кода, по которому создаются объекты. Т. е. сам по себе класс ничего не делает, но с его помощью можно создать объект и уже его использовать в работе.

Класс = Свойства + Методы

Данные внутри класса делятся на ***свойства*** и ***методы***. ***Свойства класса (они же поля)*** - это характеристики объекта класса.

Методы класса - это функции, с помощью которых можно оперировать данными класса.

Объект - это конкретный представитель класса. ***Объект класса*** и ***экземпляр класса*** - это одно и то же.

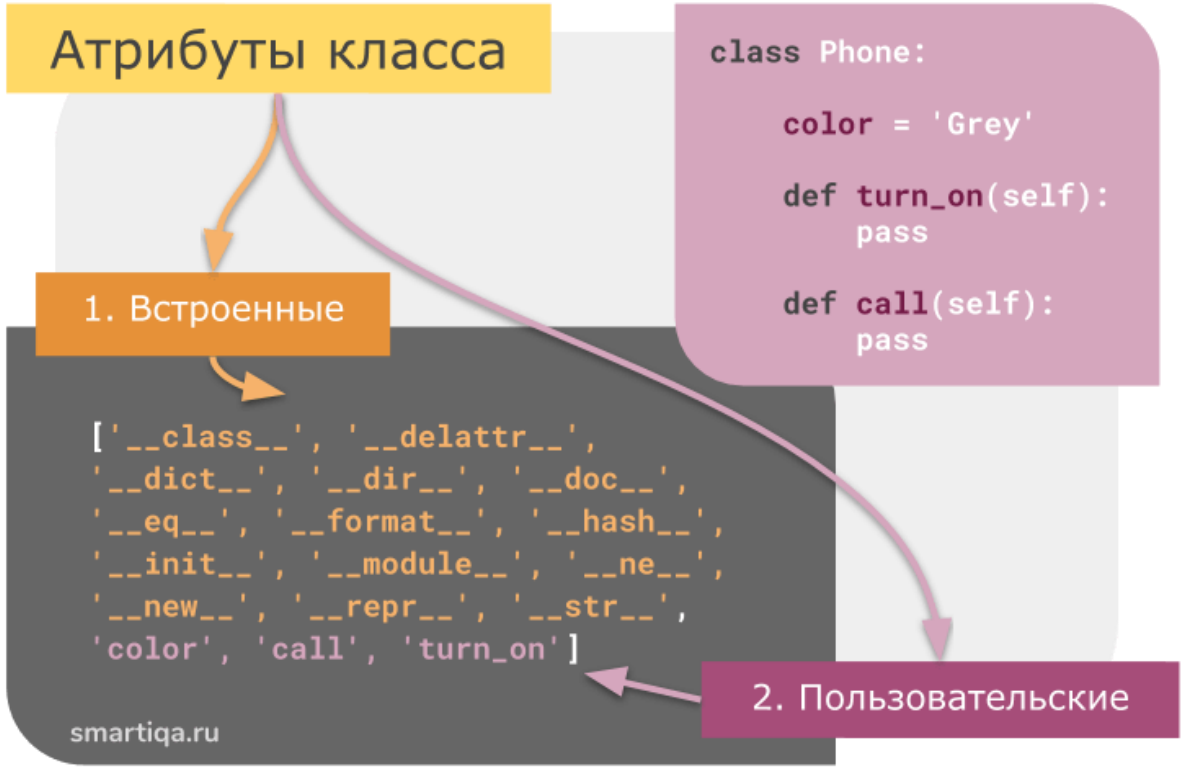
В теории ООП ***конструктор класса*** - это специальный блок инструкций, который вызывается при создании объекта.

При работе с Python может возникнуть мнение, что метод __init__(self) - это и есть конструктор, но это не совсем так. На самом деле, при создании объекта в Python вызывается метод __new__(cls, *args, **kwargs) и именно он является конструктором класса. __init__() является так называемым ***инициализатором*** класса (задавать параметры при создании).



Атрибуты класса в Python: встроенные («магические методы»)

Атрибут	Назначение	Тип
__new__(cls[, ...])	Конструктор. Создает экземпляр(объект) класса. Сам класс передается в качестве аргумента.	Функция
__init__(self[, ...])	Инициализатор. Принимает свежесозданный объект класса из конструктора.	Функция
__del__(self)	Деструктор. Вызывается при удалении объекта сборщиком мусора	Функция
__str__(self)	Возвращает строковое представление объекта.	Функция
__hash__(self)	Возвращает хэш-сумму объекта.	Функция
__setattr__(self, attr, val)	Создает новый атрибут для объекта класса с именем attr и значением val	Функция
__doc__	Документация класса.	Строка (тип str)
__dict__	Словарь, в котором хранится пространство имен класса	Словарь (тип dict)



*наследуются от базового класса **object***

Атрибуты класса в Python: встроенные

Список атрибутов класса / объекта можно получить с помощью команды **dir()**.
Первоначально в классе есть *только встроенные атрибуты*

```
class Phone:
    pass
```

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattribute__', '__gt__', '__hash__', '__init__',
'__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__',
'__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
'__str__', '__subclasshook__', '__weakref__']
```

Атрибуты класса в Python: пользовательские

Пишутся программистом во время реализации класса и дальнейшей работы с ним. Список атрибутов класса / объекта можно получить с помощью команды **dir()**.
Первоначально в классе есть *только встроенные атрибуты*

В конце списка добавились три новых пользовательских атрибута: переменная **color** и методы **turn_on()** и **call()**.

```
class Phone:

    color = 'Grey'

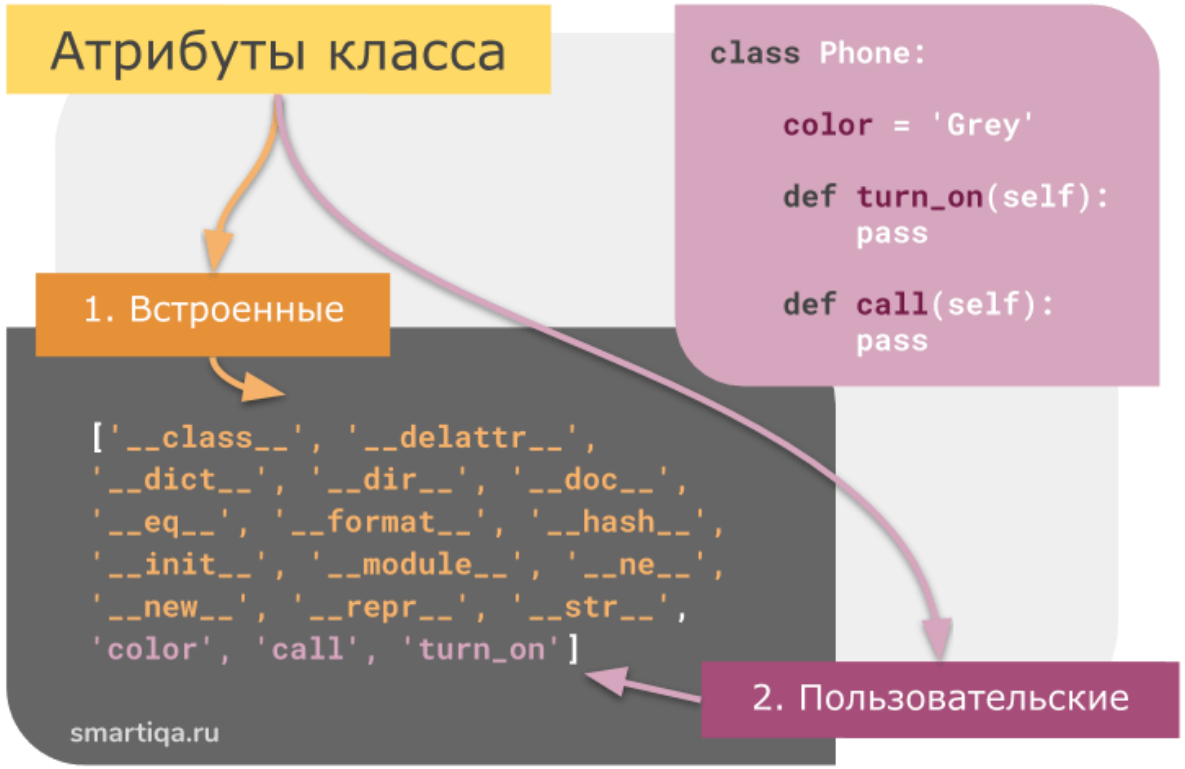
    def turn_on(self):
        pass

    def call(self):
        pass
```

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattribute__', '__gt__', '__hash__', '__init__',
'__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__',
'__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
'__str__', '__subclasshook__', '__weakref__', 'color', 'call', 'turn_on']
```

Атрибуты класса в Python

1. Атрибутами называем ***совокупность полей и методов класса / объекта***.
2. Атрибуты делятся на ***встроенные*** и ***пользовательские***.
3. Все классы в Python имеют общий родительский класс - он называется ***object***.
4. Класс object предоставляет всем своим потомкам набор ***служебных атрибутов*** (как переменных (например, __dict__ и __doc__), так и методов (например, __str__)).
5. Многие из ***служебных атрибутов*** можно переопределить внутри своего класса.
6. ***Поля и методы***, которые описываются программистом в теле класса, ***являются пользовательскими*** и добавляются в общий список атрибутов наряду со встроенными атрибутами.



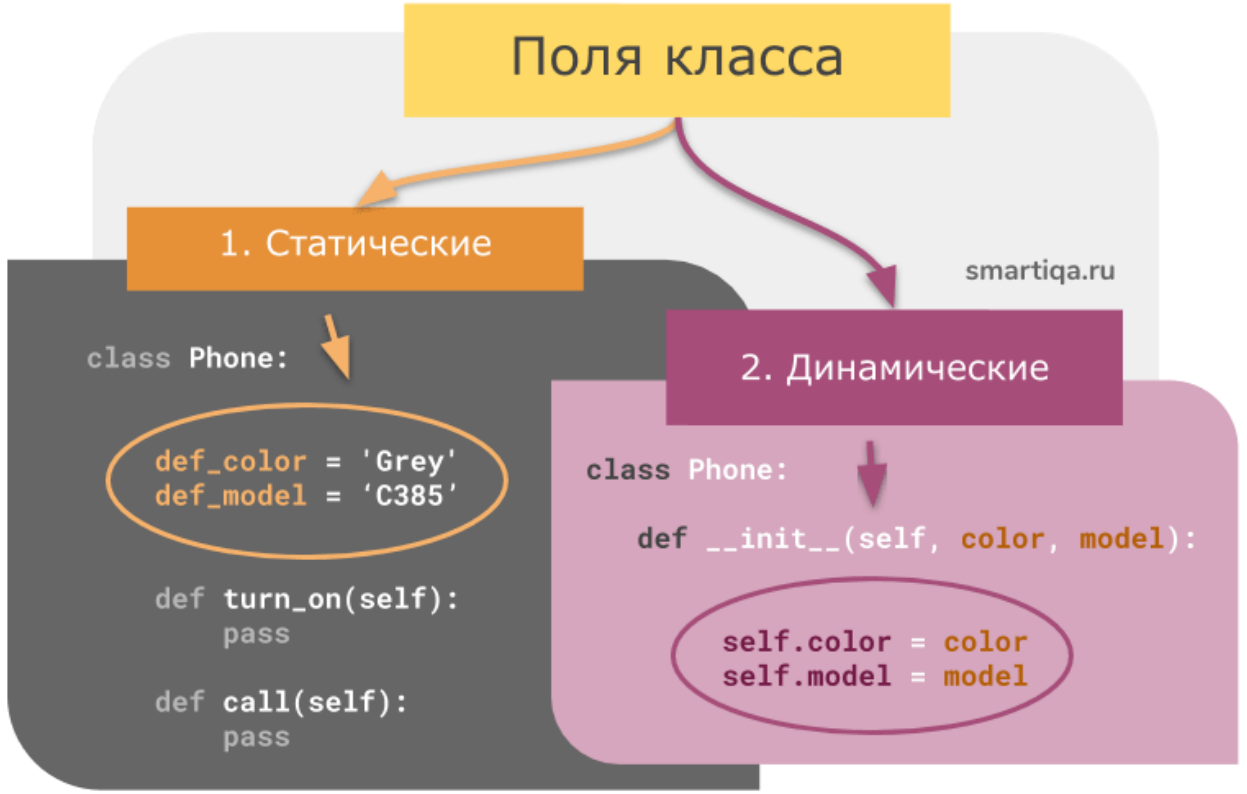
Поля (свойства) класса в Python

1. ***Статические поля (они же переменные или свойства класса)***

Это переменные, которые объявляются внутри тела класса и создаются тогда, когда создается класс. Создали класс - создалась переменная. Есть возможность получить или изменить такое свойство, просто обратившись к самому классу по его имени(экземпляр класса при этом создавать не нужно).

2. Динамические поля (переменные или свойства экземпляра класса)

Это переменные, которые создаются на уровне экземпляра класса. Нет экземпляра - нет его переменных. Для создания динамического свойства необходимо обратиться к self внутри метода. Объект класса сочетает в себе как статические атрибуты(уровня класса), так и динамические(собственные - уровня объекта).



Поля (свойства) класса в Python

1. *Статические поля (они же переменные или свойства класса)*
2. *Динамические поля (переменные или свойства экземпляра класса)*

```
class Phone:

    # Статические поля (переменные класса)
    default_color = 'Grey'
    default_model = 'C385'

    def __init__(self, color, model):
        # Динамические поля (переменные объекта)
        self.color = color
        self.model = model
```

```
# Создадим экземпляр класса Phone - телефон красного цвета модели 'I495'
>>> my_phone_red = Phone('Red', 'I495')

# Полный список атрибутов созданного экземпляра:
>>> dir(my_phone_red)
'__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__',

# Прочитаем статические поля объекта
>>> my_phone_red.default_color
'Grey'
>>> my_phone_red.default_model
'C385'

# Прочитаем динамические поля объекта
>>> my_phone_red.color
'Red'
>>> my_phone_red.model
'I495'

# Создадим еще один экземпляр класса Phone - такой же телефон, но другого цвета
>>> my_phone_blue = Phone('Blue', 'I495')

# Прочитаем динамические поля объекта
>>> my_phone_blue.color
'Blue'
>>> my_phone_blue.model
'I495'
```

Что такое self в Python?

Служебное слово self - это ссылка на текущий экземпляр класса. Как правило, эта ссылка передается в качестве первого параметра метода Python (аналог **this** в других языках).

Слово **self** не является зарезервированным. Существует некоторое соглашение, по которому первый параметр метода именуется self и передает ссылку на текущий объект, для которого этот метода был вызван.
Объект класса сочетает в себе как статические атрибуты(уровня класса), так и динамические(собственные - уровня объекта).

```
class Apple:

    # Создаем объект с общим количеством яблок 12
    def __init__(self):
        self.whole_amount = 12

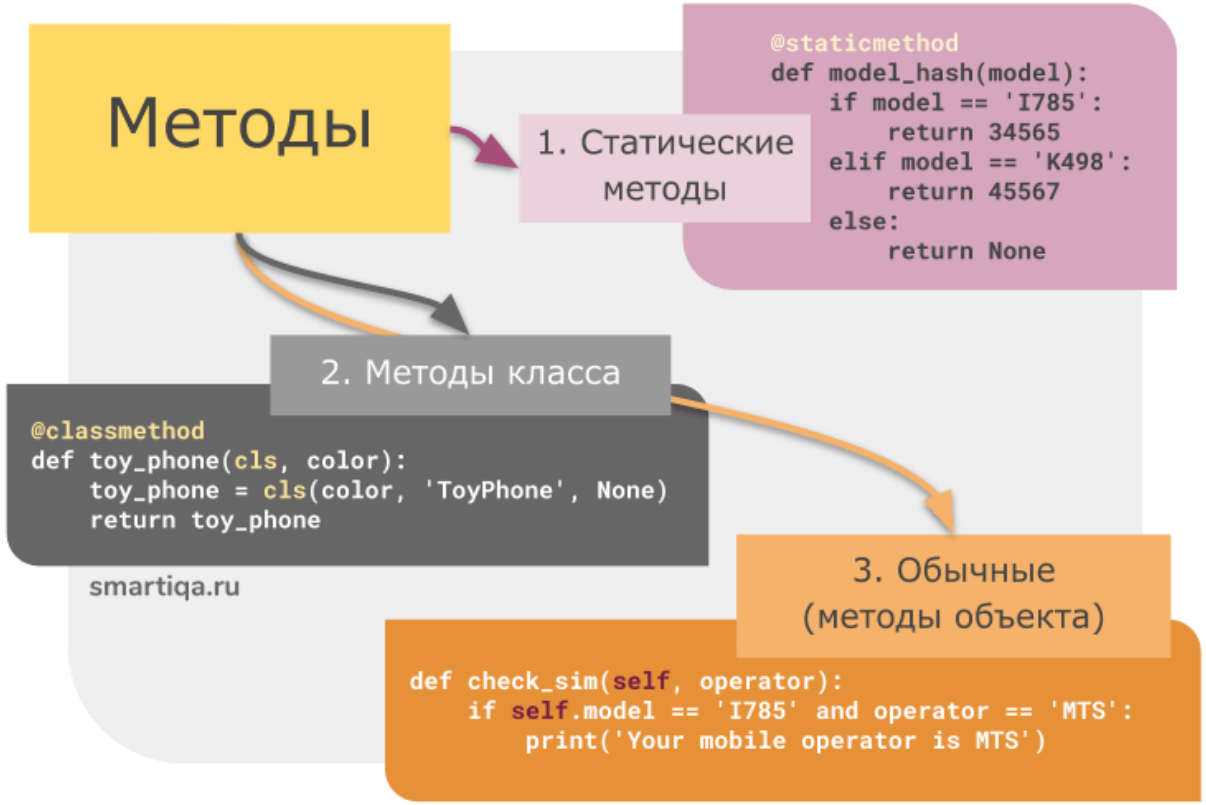
    # Съедаем часть яблок для текущего объекта
    def eat(self, number):
        self.whole_amount -= number
```

Методы (функции) класса в Python

1. Методы экземпляра класса (они же обычные методы)
Группа методов, которые становятся доступны только после создания экземпляра класса. Первым параметром такого метода является слово ***self (ссылка на объект класса)***

2. Статические методы
Обычные функции, располагаются в области видимости этого класса. Чтобы создать статический метод в Python декоратор @staticmethod. Не используется self - этому методу не важна информация об объектах класса.

3. Методы класса
Среднее между обычными методами (привязаны к объекту) и статическими методами (привязаны только к области видимости). Методы могут менять состояние самого класса, отражается на ВСЕХ экземплярах данного класса. Чтобы создать метод класса @classmethod. Первый параметр cls - ссылка на сам класс (а не на объект).



Методы (функции) класса в Python

1. Методы экземпляра класса (они же обычные методы)
2. Статические методы

```python
class Phone:

    def __init__(self, color, model):
        self.color = color
        self.model = model

    # Обычный метод
    # Первый параметр метода - self
    def check_sim(self, mobile_operator):
        if self.model == 'I785' and mobile_operator == 'MTS':
            print('Your mobile operator is MTS')
```

```python
class Phone:

    # Статический метод справочного характера
    # Возвращает хэш по номеру модели
    # self внутри метода отсутствует
    @staticmethod
    def model_hash(model):
        if model == 'I785':
            return 34565
        elif model == 'K498':
            return 45567
        else:
            return None

    # Обычный метод
    def check_sim(self, mobile_operator):
        pass
```

Методы (функции) класса в Python

3. Методы класса

```
class Phone:

    def __init__(self, color, model, os):
        self.color = color
        self.model = model
        self.os = os

    # Метод класса
    # Принимает 1) ссылку на класс Phone и 2) цвет в качестве параметров
    # Создает специфический объект класса Phone(особенность объекта в том, что это игру
    # При этом вызывается инициализатор класса Phone
    # которому в качестве аргументов мы передаем цвет и модель,
    # соответствующую созданию игрушечного телефона
    @classmethod
    def toy_phone(cls, color):
        toy_phone = cls(color, 'ToyPhone', None)
        return toy_phone

    # Статический метод
    @staticmethod
    def model_hash(model):
        pass

    # Обычный метод
    def check_sim(self, mobile_operator):
        pass
```

Краткий словарь программиста: часть 3

Private. Приватные члены класса недоступны извне - с ними можно работать только внутри класса.

Public. Публичные методы наоборот - открыты для работы снаружи и, как правило, объявляются публичными сразу по-умолчанию.

Protected. Доступ к защищенным ресурсам класса возможен только внутри этого класса и также внутри унаследованных от него классов (иными словами, внутри классов-потомков). Больше никто доступа к ним не имеет

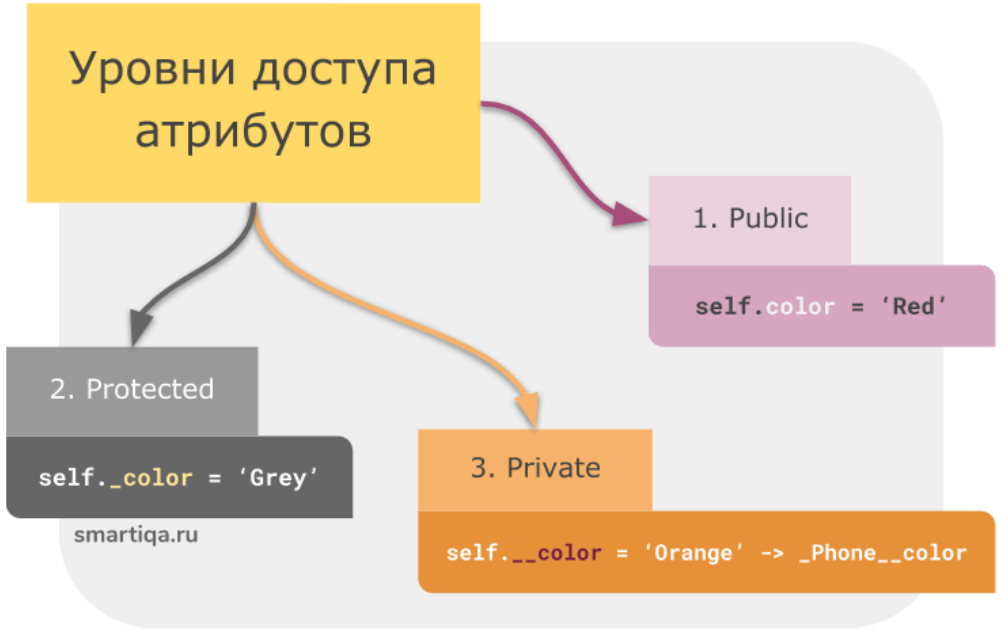


Уровни доступа атрибутов в Python

Public. Публичные методы наоборот - открыты для работы снаружи и, как правило, объявляются публичными сразу по-умолчанию. Любой член класса может быть доступен за пределами самого класса.

Private. Приватные члены класса недоступны извне - с ними можно работать только внутри класса. Если переменная/метод начинается с двух нижних подчеркиваний (__private_example), то она/он считается приватным (private).

Protected. Доступ к защищенным ресурсам класса возможен только внутри этого класса и также внутри унаследованных от него классов (иными словами, внутри классов-потомков). Больше никто доступа к ним не имеет. Если переменная/метод начинается с одного нижнего подчеркивания (_protected_example), то она/он считается защищенным (protected).



С точки зрения разграничения доступа к атрибутам класса - в Python отсутствует механизм, который мог бы запретить доступ к переменной или методу внутри класса. Вместо этого создатели Python предложили ***соглашение***

Уровни доступа атрибутов в Python

Public
Private
Protected

```python
class Phone:

    def __init__(self, color):
        # Объявляем публичное поле
        self.color = color
```

```python
# Обращаемся к свойству color
>>> phone.color
'Grey'

# Изменяем свойство color
>>> phone.color = 'Red'
>>> phone.color
'Red'
```

```python
class Phone:

    def __init__(self, color):
        # Объявляем защищенное поле _color
        self._color = color
```

В Python такой атрибут все равно будет **доступен** снаружи класса. Больше вопрос ответственности программиста

```python
# Обращаемся к защищенному свойству _color
>>> phone._color
'Grey'

# Изменяем защищенное свойство _color
>>> phone._color = 'Red'
>>> phone._color
'Red'
```

```python
class Phone:

    def __init__(self, color):
        # Объявляем приватное поле __color
        self.__color = color
```

Получить доступ к таким атрибутам напрямую нельзя (но если очень хочется, то все равно можно)

```python
# Меняем значение защищенного свойства
>>> phone._Phone__color = 'Blue'
>>> phone._Phone__color
'Blue'
```

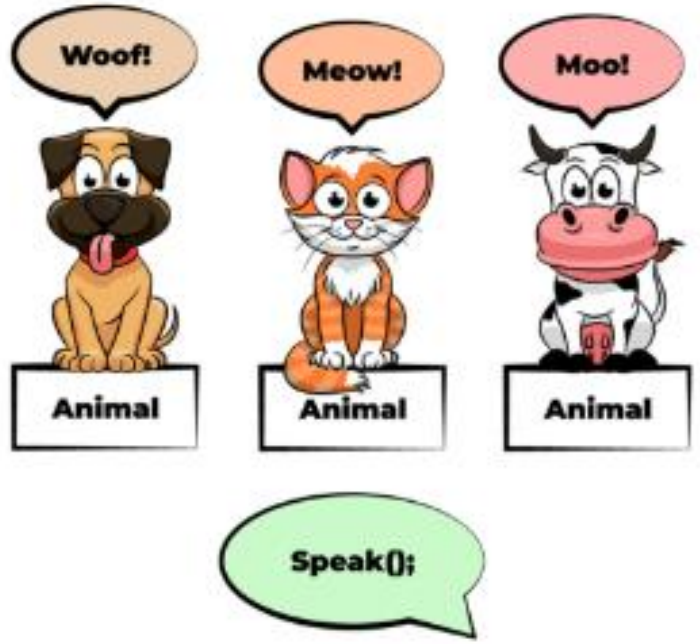
Ошибки в Python (примеры классов)

Ошибка	Пояснение	Комментарий
SyntaxError	Синтаксическая ошибка	Когда в программе обнаруживается синтаксическая ошибка, например, неправильное использование ключевых слов, неправильное расположение скобок и т. д.
NameError	Ошибка имени	Когда имя переменной или функции не определено. Это может быть связано с опечатками в имени, использованием переменной перед ее определением или обращением к несуществующей переменной.
TypeError	Ошибка типа	Когда операция применяется к объекту неправильного типа. Например, попытка складывать число и строку или вызов функции с неправильным количеством аргументов.
IndexError	Ошибка индекса	Когда индекс, используемый для доступа к элементам списка или другой последовательности, находится за пределами допустимого диапазона.
FileNotFoundError	Ошибка файла не найден	Когда программа пытается открыть несуществующий файл
ZeroDivisionError	Ошибка деления на ноль	Когда попытка деления на ноль выполняется в программе.
KeyError	Ошибка ключа	При попытке получить значение из словаря с использованием неправильного или несуществующего ключа

Распространенные типы ошибок Python или исключения (exceptions). Обрабатываем try ... except

ДЕМОНСТРАЦИЯ

- Примеры работы с классами



Принципы ООП

- абстракция
- инкапсуляция
- наследование
- полиморфизм



Принципы ООП: абстракция, ***инкапсуляция, наследование, полиморфизм***

Концепция ООП строится на основе 3 принципов. Но 0 принцип Абстракция также должен быть соблюдён

Принцип 0. **Абстракция** - объект характеризуется свойствами, которые отличают его от всех остальных объектов и при этом четко определяют его концептуальные границы.

Принцип 1. **Инкапсуляция** - сложность реализации программного компонента должна быть спрятана за его интерфейсом.

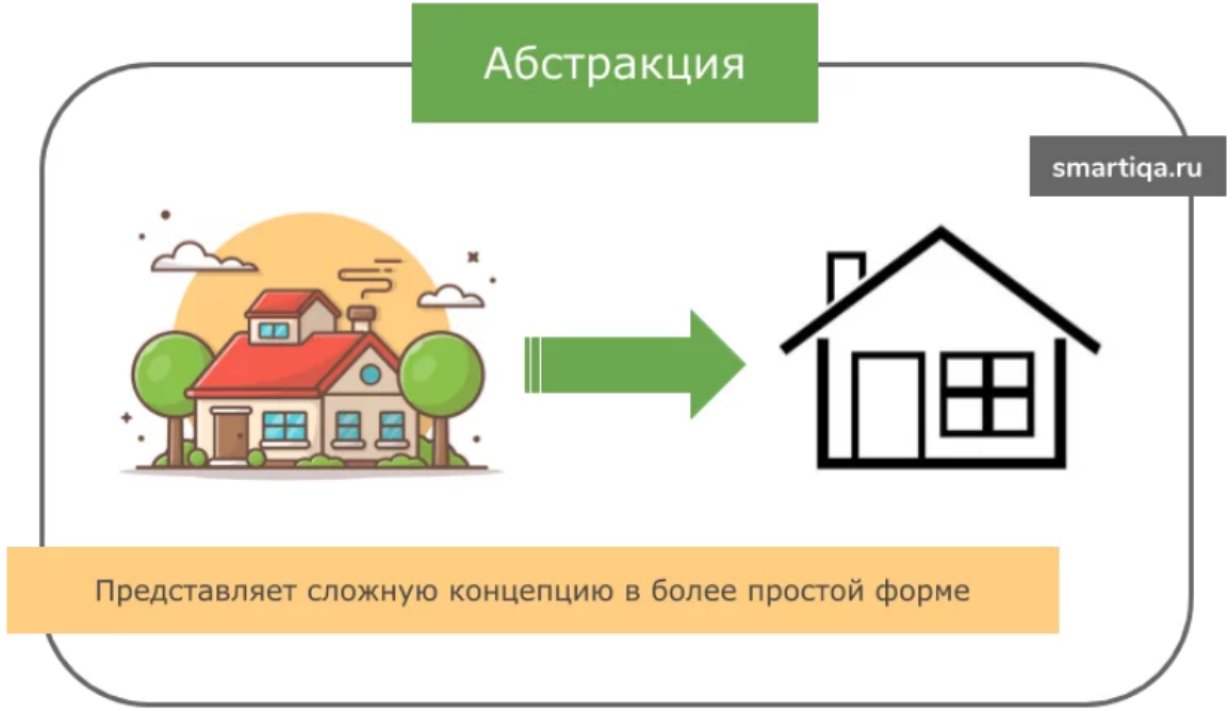
Принцип 2. **Наследование** - способ создания нового класса на основе уже существующего, при котором класс-потомок заимствует свойства и методы родительского класса и также добавляет собственные.

Принцип 3. **Полиморфизм** - это поддержка нескольких реализаций на основе общего интерфейса. **Абстрактный метод** (он же виртуальный метод) - это метод класса, реализация для которого отсутствует.

Абстракция

Абстракция - объект характеризуется свойствами, которые отличают его от всех остальных объектов и при этом четко определяют его концептуальные границы.

Позволяет:
1. Выделить главные и наиболее значимые свойства предмета. Позволяет бороться со сложностью реального мира
2. Отбросить второстепенные характеристики.



Принцип 1. Инкапсуляция

Инкапсуляция - сложность реализации программного компонента должна быть спрятана за его интерфейсом.

На что обратить внимание?

1. Отсутствует доступ к внутреннему устройству компонента.
2. Взаимодействие компонента с внешним миром осуществляется посредством интерфейса, который включает публичные методы и поля.

Для чего нужна инкапсуляция?

1. Упрощает процесс разработки, т. к. позволяет не вникать в тонкости реализации объекта.
2. Повышается надежность программ за счет внесения изменений в один из компонентов, а не во всю программу.
3. Становится более легким обмен компонентами между программами.



Принцип 2. Наследование

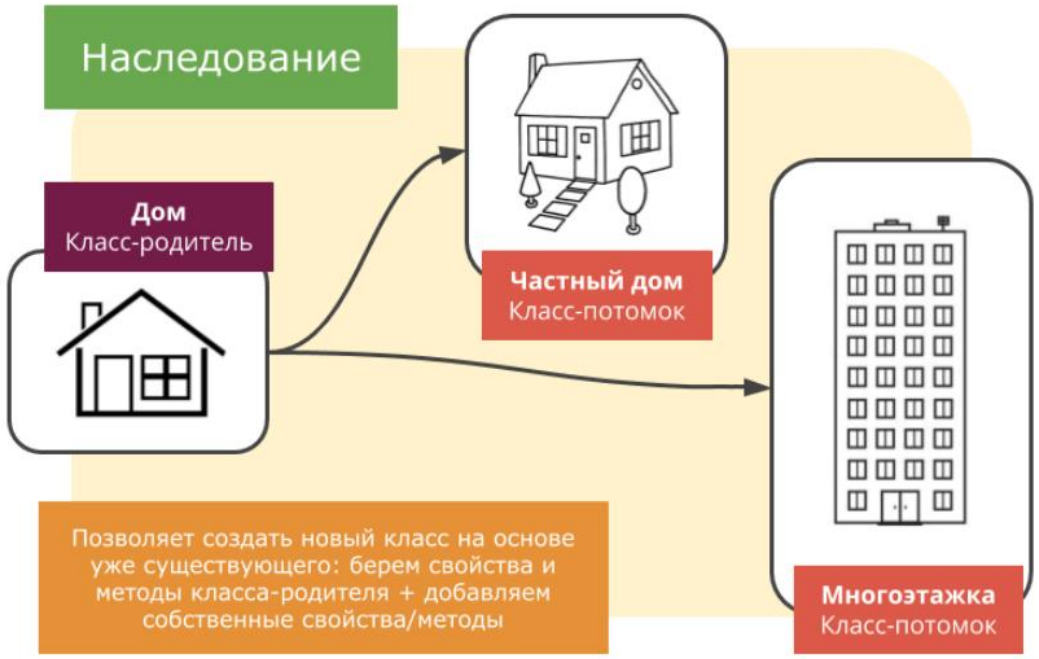
Наследование - способ создания нового класса на основе уже существующего.

На что обратить внимание?

1. Класс-потомок = Свойства и методы родителя + Собственные свойства и методы.
2. Класс-потомок автоматически наследует от родительского класса все поля и методы.
3. Класс-потомок может дополняться новыми свойствами.
4. Класс-потомок может дополняться новыми методами, а также заменять(переопределять) унаследованные методы. Переопределенный метод совпадает по названию с методом родительского класса, но функционал у него новый.

Для чего нужно наследование?

1. Дает возможность использовать код повторно.
2. Способствует быстрой разработке нового ПО на основе уже существующих открытых классов.
3. Наследование позволяет делать процесс написания кода более простым.



Многоэтажный дом

СВОЙСТВА	МЕТОДЫ
1) Тип фундамента (УНАСЛЕДОВАНО)	1) Построить (УНАСЛЕДОВАНО)
2) Материал крыши (УНАСЛЕДОВАНО)	2) Отремонтировать (УНАСЛЕДОВАНО)
3) Количество окон (УНАСЛЕДОВАНО)	3) Заселить (УНАСЛЕДОВАНО)
4) Количество дверей (УНАСЛЕДОВАНО)	4) Снести (УНАСЛЕДОВАНО)
5) Количество квартир	5) Выбрать управляющую компанию
6) Количество подъездов	6) Организовать собрание жильцов
7) Наличие коммерческой недвижимости	7) Нанять дворника

Принцип 2. Наследование

Для того, чтобы в Python создать новый класс с помощью механизма наследования, необходимо воспользоваться следующим синтаксисом:

```
class <имя_нового_класса>(<имя_родителя>):
```

```
# Унаследованный класс
class MobilePhone(Phone):

    # Добавляем новое свойство battery
    def __init__(self):
        super().__init__()
        self.battery = 0

# Заряжаем телефон на величину переданного значения
def charge(self, num):
    self.battery = num
    print(f'Charging battery up to ... {self.battery}%')
```

Класс **Phone** (Телефон), у которого есть одно свойство **is_on** и три метода:

1. Инициализатор: **__init__()**
2. Включение: **turn_on()**
3. Звонок: **call()**

```
# Родительский класс
class Phone:

    # Инициализатор
    def __init__(self):
        self.is_on = False

    # Включаем телефон
    def turn_on(self):
        self.is_on = True

    # Если телефон включен, делаем звонок
    def call(self):
        if self.is_on:
            print('Making call...')
```

Что такое super?

Главная задача super() метода - дать возможность наследнику **обратиться к родительскому классу**.

В классе-родителе Phone есть свой инициализатор. Когда в потомке MobilePhone создаем инициализатор - мы его **перегружаем**.
Перегрузка - заменяем родительского метода __init__() собственным одноименным методом.

Чтобы потери не произошло:
1. Внутри инициализатора класса-наследника вызвать инициализатор родителя (для этого вызываем метод super().__init__())
2. Добавить новый функционал

Метод __init__() класса MobilePhone:

```
def __init__(self):
    super().__init__()
    self.battery = 0
```

Вызывать родительский метод необходимо в первую очередь.

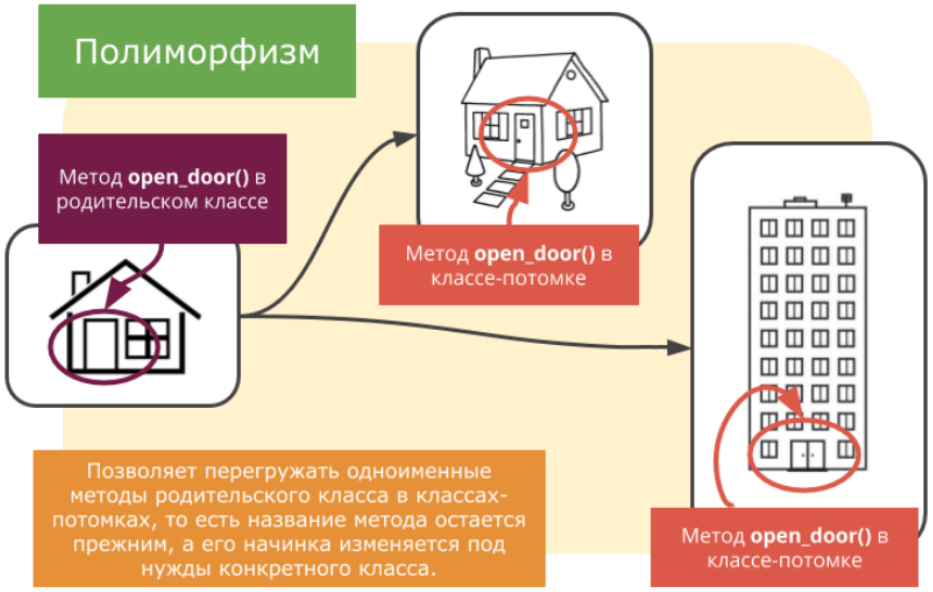
Принцип 3. Полиморфизм

Полиморфизм - это поддержка нескольких реализаций на основе общего интерфейса. Позволяет перегружать одноименные методы родительского класса в классах-потомках.

Также для понимания работы этого принципа важным является понятие абстрактного метода:

Абстрактный метод (он же виртуальный метод) - это метод класса, реализация для которого отсутствует.

За одинаковым названием могут скрываться методы с совершенно разным функционалом, который в каждом конкретном случае соответствует нуждам класса, к которому он относится.



1. В родительском классе(класс **Дом**) создают пустой метод(**Построить()**) и делают его абстрактным.
2. В классах-потомках создают одноименные методы с реализацией. К примеру, для строительства **Многоквартирного дома** необходимо задействовать башенный кран, а **Частный дом** можно построить и собственными силами.
3. Получаем метод с одним и тем же именем, который встречается во всех классах. В родительском - только интерфейс, реализация отсутствует. В классах-потомках - интерфейс и реализация. Реализация в потомках становится обязательной.
4. Достаточно просто вызвать метод **Построить()**. В момент исполнения программы, когда класс объекта станет известен, будет вызвана необходимая реализация метода **Построить()**.

Принцип 3. Полиморфизм

```python
# Родительский класс
class Phone:

    def __init__(self):
        self.is_on = False

    def turn_on(self):
        pass

    def call(self):
        pass

    # Метод, который выводит короткую сводку по классу Phone
    def info(self):
        print(f'Class name: {Phone.__name__}')
        print(f'If phone is ON: {self.is_on}')
```

```python
# Унаследованный класс
class MobilePhone(Phone):

    def __init__(self):
        super().__init__()
        self.battery = 0

    # Такой же метод, который выводит короткую сводку по классу MobilePhone
    # Обратите внимание, что названия у методов совпадают - оба метода называются info
    # Однако их содержимое различается
    def info(self):
        print(f'Class name: {MobilePhone.__name__}')
        print(f'If mobile phone is ON: {self.is_on}')
        print(f'Battery level: {self.battery}')
```

```python
# Демонстрационная функция

# Создаем список из классов
# В цикле перебираем список и для каждого элемента списка(а элемент - это класс)
# Создаем объект и вызываем метод info()
# Главная особенность: запись object.info() не дает информацию об объекте, для которог
# Это может быть объект класса Phone, а может - объект класса MobilePhone
# И только в момент исполнения кода становится ясно, для какого именно объекта нужно вы
def show_polymorphism():
    for item in [Phone, MobilePhone]:
        print('-------')
        object = item()
        object.info()
```

Краткий словарь программиста: часть 2

Полиморфизм - свойство системы, позволяющее использовать объекты с одинаковым интерфейсом без информации о типе и внутренней структуре объекта

Инкапсуляция - ограничение доступа к составляющим объект компонентам (методам и переменным). Инкапсуляция делает некоторые из компонент доступными только внутри класса.

Инкапсуляция в Python работает лишь на уровне соглашения между программистами о том, какие атрибуты являются общедоступными, а какие — внутренними

Наследование - свойство системы, позволяющее описать новый класс на основе уже существующего с частично или полностью заимствованной функциональностью.

Класс, от которого производится наследование, называется ***базовым, родительским или суперклассом.*** Новый класс — ***потомком, наследником, дочерним или производным классом.***



Причём здесь ООП?

Проблема с функциями (если не используем свои **кастомные** классы вообще) в том, что все параметры, которые мы им собираемся передавать, будут на самом низком уровне абстракции: DataFrame-ы, словари, списки.

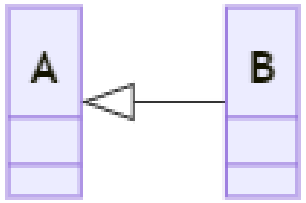
При росте проекта, когда увеличивается количество сценариев и соответствующих параметров, которые хотим покрыть, начинает распухать количество аргументов в функциях.

С использованием классов можно (в идеале) выстроить уровни абстракции, то есть, собственно, уровни классов.

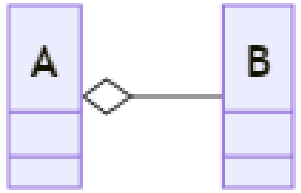
Хороший ООП код — это максимально скучная пьеса.
Если код организован подобным образом, снижается его сложность.
Под сложностью здесь понимаем сложность восприятия: насколько легко его можно понять при прочтении, насколько легко вносить правки. Есть замечательная популярная статья (The Grug Brained Developer), где очень доступно описана необходимость бороться со сожностью

Отношения между классами

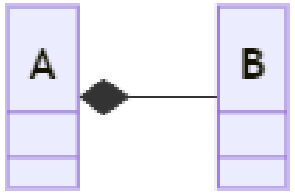
Наследование: класс B унаследован от A.



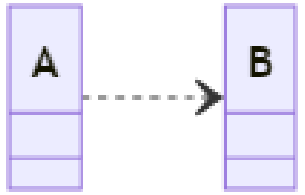
Агрегация: объект класса A содержит объекты класса B.



Композиция: объект класса A состоит из объектов класса B (лишен смысла без них).



Ассоциация: объект A использует объект B.



Отношения между классами

Агрегация предполагает, что объект одного класса может содержать (агрегировать) объекты другого класса. При этом оба класса остаются независимыми.

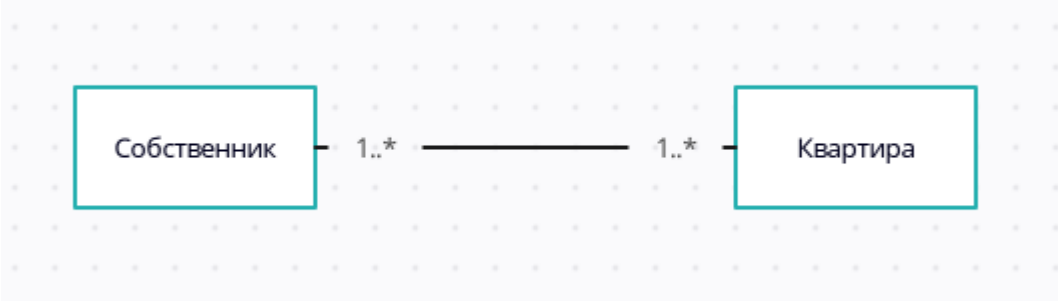


Композиция несколько напоминает агрегацию, но существенное отличие. При композиции экземпляры второго класса не могут существовать самостоятельно. Оценка не может существовать сама по себе, он жестко привязана к ученику.



Отношения между классами

Ассоциация. Часто встречающаяся связь. Реализуется, когда объекты одного класса могут быть связаны с объектами другого класса через переменные объекта. Ниже представлен типичный пример ассоциативной связи, её подвида "многие ко многим". Собственник (объект) может иметь список ссылок на объекты класса Квартиры, владельцем которых он является.

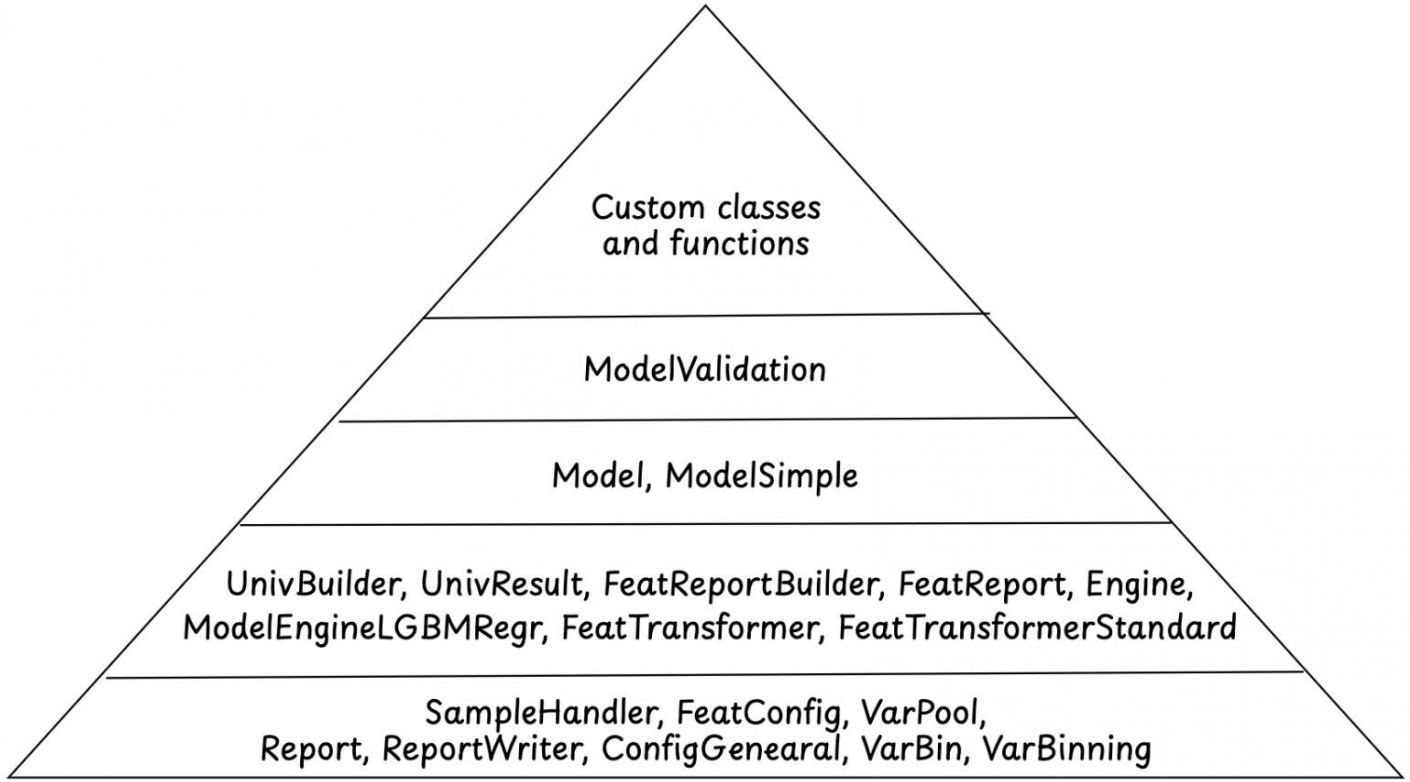


Зависимость последний тип связи между классами. Выражается он в том, выполнение методов одного класса зависит от объектов другого класса. Магазин делает заказ на склад. Выполнение заказа зависит от того, есть в наличие на складе тот или иной продукт и его количество. Т.е. метод объекта Магазин должен использовать объект класса Склад, чтобы узнать наличие на складе тех или иных продуктов.



ООП и классы в ML

Чем ниже уровень абстракции, тем больше классов, кода, и тем больше можно подготовить тестов и больше переиспользовать.
Чем выше уровень, тем больше вероятность, что придётся не использовать класс as is, а частично переписать или вообще написать новый (хорошо, если получится опираться на некоторый интерфейс, который следует имплементировать, т.е. отнаследоваться от него).



Иногда достаточно 1 класса

Python-класс, который предоставляет интерфейс для загрузки данных, обучения моделей, оценки их качества и логирования результатов экспериментов.

```python
class MLExperimentManager:
    data = None
    X_train = X_test = y_train = y_test = None
    model = None
    experiments_log = None
    log_path = 'experiments_log.csv'

    def load_data(self, dataset): ...

    def train_model(self, classifier='RandomForest'): ...

    def evaluate_model(self, metric='accuracy'): ...

    def log_experiment_to_csv(self, experiment_id, model_params, data_path, result, metric):

    # актуально только для бинарной классификации
    def plot_roc_curves(self): ...

    def plot_results(self): ...
```

Иногда достаточно 1 класса

Python-класс, который предоставляет интерфейс для загрузки данных, обучения моделей, оценки их качества и логирования результатов экспериментов.

load_data. Метод отвечает за загрузку данных в формате Parquet/CSV и предоставляет интерфейс для их предварительной обработки. Делит данные на обучающий и тестовый наборы. Модель позволяет проводить тесты на данных, находящихся за горизонтом обучения — то есть на выборке out-of-time (OOT). Иными словами, её работу можно проверять на данных, которые она вообще не видела.

train_model. Метод предназначен для обучения модели. Параметры выбранной модели можно оптимизировать.

evaluate_model. Метод для оценки качества модели.

log_experiment_to_csv. Метод отвечает за сохранение результатов эксперимента.

plot_roc_curves. Метод для визуализации фактических и предсказанных результатов.

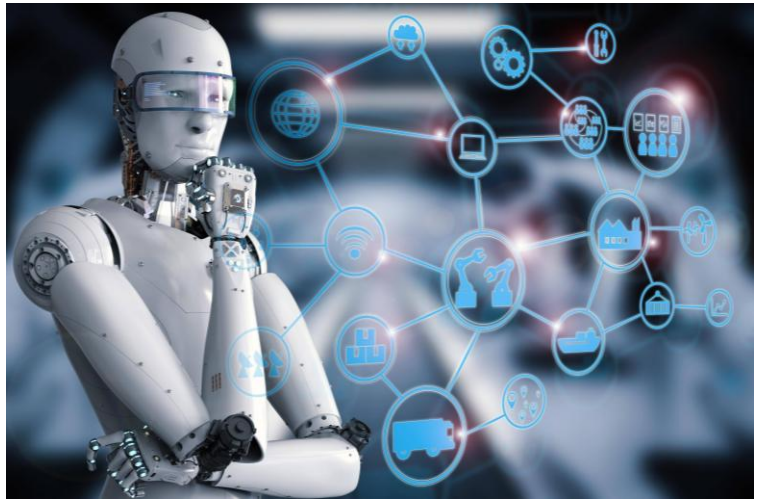
Продолжение...

- Решение задач с применением принципов ООП на Python



Полезные ссылки

1. ООП. Классы и объекты. URL: https://www.dmitrymakarov.ru/python/oop-12/
2. Когда достаточно простого класса Python — взять и начать управлять ML-экспериментами. URL: https://habr.com/ru/companies/pgk/articles/812821/
3. OOP for Machine Learning. URL: https://thecleverprogrammer.com/2020/08/08/oop-for-machine-learning/
4. ООП для типовых ML задач. URL: https://habr.com/ru/companies/alfa/articles/851154/
5. Understanding Object-Oriented Programming Through Machine Learning. URL: https://dziganto.github.io/classes/data%20science/linear%20regression/machine%20learning/object-oriented%20programming/python/Understanding-Object-Oriented-Programming-Through-Machine-Learning/
6. Принципы ООП. Классы, объекты, поля и методы. Уровни доступа. URL: https://smartiqa.ru/courses/python/lesson-6
7. Объектно-ориентированное программирование на языке Python. URL: https://cmp.phys.msu.ru/sites/default/files/ООП_на_Python_Учебное%20пособие_var7.pdf
8. Объектно-ориентированное программирование и классы. URL: https://www.yuripetrov.ru/edu/python/ch_10_01.html



Спасибо за внимание!



Контакты

- Корнеева Елена
- https://t.me/Allyonzy